第28章 矩陣運算：詳解

28-E01 兩次三角代入

推導

消去乘數$m_{21}=3$，$L=[1\ 0;3\ 1]$、$U=[1\ 2;0\ -2]$，$P=I$。$Ly=b$依序給$y_1=5$、$y_2=11-3\cdot5=-4$；$Ux=y$反向給$x_2=(-4)/(-2)=2$、$x_1=5-2\cdot2=1$。代回原A，第一列1+4=5，第二列3+8=11。

另一矩陣[0 1;1 0]的平方為I，所以它可逆，逆就是自己。其首pivot0只代表這個不置換的消去次序不能直接做；交換兩個row後PA=I即可取L=U=I。

結論與檢核

$x=(1,2)^{T}$，兩階段各維持已解列的方程。零對角不是奇異的充分條件；只有當前active column全零等條件才支援LUP的singular判斷。

28-E02 Schur 補與矩陣逆

推導

B=1,C=1,D=2,E=5，所以$S=5-2\cdot1\cdot1=3$。公式四格依序為
$1+1\cdot1\cdot(1/3)\cdot2\cdot1=5/3$，$-1\cdot1\cdot(1/3)=-1/3$，$-(1/3)\cdot2\cdot1=-2/3$，$1/3$。故$A^{-1}=[5/3\ -1/3;-2/3\ 1/3]$。

AA^{-1}四格為$5/3-2/3=1$，$-1/3+1/3=0$，$10/3-10/3=0$，$-2/3+5/3=1$。若B奇異，B^{-1}根本未定義，不能先套再希望整塊抵消。例如A=[0 1;1 0]可逆，B=0卻不可逆，需要不同置換或其他分塊。

結論與檢核

S=3、det A=3、逆如上。一般矩陣的block公式保留乘法順序；此題blocks恰為scalar才可交換，不能用此特例概括矩陣。

28-E03 非零殘差的直線擬合

推導

$A=[1\ 0;1\ 1;1\ 2]$、$b=(0,2,1)^{T}$，$A^{T}A=[3\ 3;3\ 5]$、$A^{T}b=(3,4)^{T}$。兩式相減得$2m=1$，再代入$3c+3/2=3$，得$c=m=1/2$。預測$p=(1/2,1,3/2)$，$r=b-p=(-1/2,1,-1/2)$。

$A^{T}r=(-1/2+1-1/2,0\cdot(-1/2)+1\cdot1+2\cdot(-1/2))=(0,0)$，平方和$1/4+1+1/4=3/2$。這不是三個散點到擬合線的最短距離平方和，而是固定各t後的y誤差平方和。

改b=(1,2,3)時A完全未變，columns仍獨立，rank仍2。新的唯一係數$c=m=1$，殘差0。資料是否共線決定此直線模型能否精確擬合，不決定只由t與basis所建A的rank。

第28章 矩陣運算：詳解 （續）

結論與檢核

原最小值3/2，改資料後最小值0，但兩次Gram完全相同且det=6>0。不要把「非零殘差」與「唯一解」的條件混在一起。

28-E04 後續置換也要交換舊 L

推導

第一column最大abs為2，交換row1、3，p=(3,2,1)。乘數row2為1/2、row3為0，active 2×2塊成[−1/2 −2;2 1]。第二column最大abs為2，交換目前row2、3，p=(3,1,2)，舊第一column乘數變為row2的0、row3的1/2。新乘數$m_{32}=-1/4$，最後pivot−7/4。

L=[1 0 0;0 1 0;1/2 −1/4 1]，U=[2 3 4;0 2 1;0 0 −7/4]。LU rows=[2,3,4]、[0,2,1]、[1,1,0]，等於依原row3、1、2重排的PA。

Pb=(7,0,0)，前向y=(7,0,−7/2)，後向x=(1,−1,2)。原A乘回為(0,0,7)。若錯留$L_{21}=1/2$，錯誤LU第二row為(1/2)・(2,3,4)+(0,2,1)=(1,7/2,3)，但PA第二row應為(0,2,1)，已不相等。

結論與檢核

兩個row swaps都須包含先前乘數；p與Pb也必同步。正確det A為2・2・(−7/4)=−7，兩次swap的符號相乘為正，與原det一致。

28-E05 兩向複雜度歸約

推導

T乘候選逆的對角blocks皆I；(1,2)為−B+B=0，(2,3)為−C+C=0，(1,3)為BC+B・(−C)+0=0，其餘下三角皆0。因此逆的右上確為BC，全程沒有改成CB。

建T需$\Theta(n^2)$寫入，須$I(n)=\Omega(n^2)$才能吸收。求逆的是3n維，不是n維，$I(3n)=O(I(n))$才能保證總$O(I(n))$，任意不規則成本函數不能省略此假設。

反向第ℓ層有2^{ℓ}個子問題，各$M(n/2^{\ell})\le cM(n)$，故該層$O((2c)^{\ell}M(n))$；$2c<1$使和不超過常數$M(n)$，加葉成本亦被吸收。先對SPD用可逆leading block與SPD Schur執行兩個遞迴逆。對一般實可逆A，$G=A^{T}A$為SPD，且$G^{-1}A^{T}\cdot A=I$，所以$A^{-1}=G^{-1}A^{T}$，兩次額外product均$O(M(n))$。

結論與檢核

兩向歸約是維度與成本條件完整的精確算術結論。形成$A^{T}A$會平方條件數，因此這個理論步驟本身不是浮點線性系統的預設實作方式。

第28章 矩陣運算：詳解 （續）

28-E06 SPD 與 Cholesky

推導

$C_{11}=\sqrt{4}=2$，$C_{21}=2/2=1$，$C_{31}=2/2=1$；$C_{22}=\sqrt{(5-1)}=2$，$C_{32}=(1-1\cdot 1)/2=0$；$C_{33}=\sqrt{(3-1^2-0^2)}=\sqrt{2}$。C=[2 0 0;1 2 0;1 0 $\sqrt{2}$]。CC^T的rows為[4,2,2]、[2,5,1]、[2,1,3]，等於A；C可逆，任意$z\neq 0$有$z^TAz=\|C^Tz\|^2>0$。

首pivot4的Schur為 $\begin{bmatrix}5 & 1\\ 1 & 3\end{bmatrix}-\begin{bmatrix}2\\ 2\end{bmatrix}\frac{1}{4}\begin{bmatrix}2 & 2\end{bmatrix}=\begin{bmatrix}4 & 0\\ 0 & 2\end{bmatrix}$，對非零(u,v)二次型$4u^2+2v^2>0$。這也解釋後面兩個正pivot4、2。

反例B=[1 2;2 1]的兩個對角都正，但取z=(1,−1)，$z^TBz=1-2-2+1=-2<0$，所以非SPD；直接消去第二pivot$1-2\cdot 2=-3$也暴露問題。

結論與檢核

SPD須量化所有非零向量，不能只看原對角。此例A確SPD，正定Schur使Cholesky依次合法，不需要row pivoting。

28-E07 Rank 缺失與誤差空間

推導

所有預測Ax都為$s\cdot(1,1,1)$，其中$s=c_1+c_2$。平方誤差$s^2+(2-s)^2+(1-s)^2=3(s-1)^2+2$，最小在$s=1$。因此全部係數$(c_1,c_2)=(t,1-t)$，唯一預測(1,1,1)，r=(−1,1,0)，平方和2，$A^Tr=(0,0)$。

係數差落在null A時s不變，所以不同係數可同樣好；若改到sum=1+δ，平方誤差變$2+3\delta^2$，$\delta\neq 0$就嚴格增加。rank只有1，正規方程只給同一條$c_1+c_2=1$，不能聲稱Gram可逆。

diag$(1,10^{-3})$的最大／最小奇異值為1與10^{-3}，$\kappa_2=10^3$；Gram=diag$(1,10^{-6})$，$\kappa_2=10^6$。這衡量某些方向擾動可能被放大，並不表示算法執行了10^6次操作；分解仍按n的維度有立方算術界。

結論與檢核

係數不唯一、預測與正交殘差仍唯一。敏感度、唯一性與時間複雜度是不同問題，需各用正確條件判定。

28-E08 同一向量，兩套座標

先算 $\|p\|^2=1/4+1+9/4=7/2$、$\|r\|^2=1/4+1+1/4=3/2$，而 $p^Tr=-1/4+1-3/4=0$。因此

$$u=\frac{(1,2,3)^T}{\sqrt{14}},\qquad v=\frac{(-1,2,-1)^T}{\sqrt{6}}.$$

第28章 矩陣運算：詳解 （續）

兩者長度皆1且內積0。由b=p+r，截面座標為

$$[b]_{(u,v)} = (u^T b, v^T b)^T = \left(\sqrt{7/2}, \sqrt{3/2}\right)^T.$$

還原得到 $\sqrt{7/2}\,u + \sqrt{3/2}\,v = p + r = (0,2,1)^T$。截面座標有兩個分量，原始座標有三個；它們描述同一向量，不能把兩套分量直接混用。

因p=A(1/2,1/2)ᵀ∈Col(A)，span{p}包含於交集。反向，設z=αp+βr∈Col(A)。由AᵀrT=0得r⊥Col(A)，所以0=rᵀz=β||r||²，故β=0，z∈span{p}。兩方向合併便是E∩Col(A)=span{p}。A的兩個columns獨立，Col(A)仍為二維；圖只畫出它與另一個二維平面的交線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第28章 矩陣運算：詳解 (續)

原創驗算程式附錄: least_squares_check.py

```
"""Check the exact least-squares example and its orthogonality condition."""


def mat_vec(a, v):
    return [sum(row[j] * v[j] for j in range(len(v))) for row in a]


def dot(u, v):
    return sum(x * y for x, y in zip(u, v))


def solve_2x2(a, b):
    det = a[0][0] * a[1][1] - a[0][1] * a[1][0]
    assert det != 0
    return [
        (b[0] * a[1][1] - a[0][1] * b[1]) / det,
        (a[0][0] * b[1] - b[0] * a[1][0]) / det,
    ]


def run():
    a = [[1, 0], [1, 1], [1, 2]]
    b = [0, 2, 1]
    ata = [[dot([row[i] for row in a], [row[j] for row in a]) for j in range(2)] for i in
range(2)]
    atb = [dot([row[i] for row in a], b) for i in range(2)]
    assert ata == [[3, 3], [3, 5]]
    assert atb == [3, 4]
    solution = solve_2x2(ata, atb)
    assert solution == [0.5, 0.5]
    prediction = mat_vec(a, solution)
    residual = [target - fitted for target, fitted in zip(b, prediction)]
    assert prediction == [0.5, 1.0, 1.5]
    assert residual == [-0.5, 1.0, -0.5]
    assert [dot([row[i] for row in a], residual) for i in range(2)] == [0, 0]
    assert round(dot(residual, residual), 12) == 1.5
    print("PASS least-squares intercept=0.5 slope=0.5 residual=(-0.5,1,-0.5) AtR=0")
    return solution, residual


if __name__ == "__main__":
    run()
```

第28章 矩陣運算：詳解 （續）

原創驗算程式附錄: matrix_operations_checks.py

```
"""Exact rational matrix identities, independently checked small determinants."""
from fractions import Fraction as F
from itertools import product,permutations
from math import prod
import random

SAMPLE=[[0,2,1],[1,1,0],[2,3,4]]


def transpose(a):return [list(row) for row in zip(*a)]
def identity(n):return [[F(i==j) for j in range(n)] for i in range(n)]
def multiply(a,b):return [[sum((x*y for x,y in zip(row,col)),F(0)) for col in zip(*b)] for
row in a]
def matvec(a,x):return [sum((v*w for v,w in zip(row,x)),F(0)) for row in a]
def subtract(a,b):return [[x-y for x,y in zip(r,s)] for r,s in zip(a,b)]


def determinant(a):
    n=len(a);answer=F(0)
    for p in permutations(range(n)):
        inversions=sum(p[i]>p[j] for i in range(n) for j in range(i+1,n))
        answer+=(-1)**inversions*prod((a[i][p[i]] for i in range(n)),start=F(1))
    return answer


def lup(a):
    n=len(a);work=[[F(x) for x in row] for row in a];order=list(range(n));trace=[];swaps=0
    for k in range(n):
        pivot=max(range(k,n),key=lambda i:abs(work[i][k]))
        if work[pivot][k]==0:raise ValueError('singular matrix')
        if pivot!=k:
            work[k],work[pivot]=work[pivot],work[k]
            order[k],order[pivot]=order[pivot],order[k];swaps+=1
        for i in range(k+1,n):
            work[i][k]/=work[k][k]
            for j in range(k+1,n):work[i][j]-=work[i][k]*work[k][j]
        trace.append(([row[:] for row in work],order[:]))
    lower=[[F(i==j) if i<=j else work[i][j] for j in range(n)] for i in range(n)]
```

第28章 矩陣運算：詳解 （續）

```
    upper=[[work[i][j] if i<=j else F(0) for j in range(n)] for i in range(n)]
    return lower,upper,order,trace,swaps


def solve(factor,b):
    lower,upper,order,_,_=factor;n=len(b);y=[F(0)]*n;x=y[:]
    for i in range(n):y[i]=F(b[order[i]])-sum((lower[i][j]*y[j] for j in range(i)),F(0))
    for i in range(n-1,-1,-1):x[i]=(y[i]-sum((upper[i][j]*x[j] for j in
range(i+1,n)),F(0)))/upper[i][i]
    return x,y


def inverse(a):
    n=len(a);factor=lup(a);columns=[solve(factor,[int(i==j) for i in range(n)])[0] for j in
range(n)]
    return transpose(columns)


def ldlt(a):
    n=len(a);lower=identity(n);diagonal=[F(0)]*n
    assert a==transpose(a)
    for j in range(n):
        diagonal[j]=a[j][j]-sum((lower[j][k]**2*diagonal[k] for k in range(j)),F(0))
        if diagonal[j]<=0:raise ValueError('not positive definite')
        for i in range(j+1,n):
            lower[i][j]=(a[i][j]-sum((lower[i][k]*lower[j][k]*diagonal[k] for k in
range(j)),F(0)))/diagonal[j]
    return lower,[[diagonal[i] if i==j else F(0) for j in range(n)] for i in range(n)]


def schur(a,cut):
    b=[row[:cut] for row in a[:cut]];c=[row[cut:] for row in a[:cut]]
    d=[row[:cut] for row in a[cut:]];e=[row[cut:] for row in a[cut:]]
    return subtract(e,multiply(multiply(d,inverse(b)),c))


def least_squares(a,b):
    at=transpose(a);gram=multiply(at,a);rhs=matvec(at,b)
    x,y=solve(lup(gram),rhs);prediction=matvec(a,x);residual=[F(u)-v for u,v in
zip(b,prediction)]
```

第28章 矩陣運算：詳解 （續）

```
    return x,prediction,residual,gram,rhs


def main():
    count=0;regular=0;inversions=0
    for n in range(1,4):
        for values in product([-1,0,1],repeat=n*n):
            a=[list(values[i*n:(i+1)*n]) for i in range(n)];det=determinant(a)
            try:factor=lup(a)
            except ValueError:
                assert det==0;count+=1;continue
            assert det!=0;regular+=1
            l,u,p,trace,swaps=factor
            assert multiply(l,u)==[a[i] for i in p]
            assert (-1)**swaps*prod((u[i][i] for i in range(n)),start=F(1))==det
            expected=[F(i+1) for i in range(n)];b=matvec(a,expected)
            assert solve(factor,b)[0]==expected
            if count%17==0 or n<3:
                inv=inverse(a);assert
multiply(a,inv)==identity(n)==multiply(inv,a);inversions+=1
            count+=1
    factor=lup(SAMPLE);l,u,p,trace,swaps=factor
    assert p==[2,0,1] and swaps==2
    assert l==[[1,0,0],[0,1,0],[F(1,2),F(-1,4),1]]
    assert u==[[2,3,4],[0,2,1],[0,0,F(-7,4)]]
    assert solve(factor,[0,0,7])==([1,-1,2],[7,0,F(-7,2)])
    rng=random.Random(28);spd_cases=0;least_cases=0
    for n in range(1,7):
        for _ in range(40):
            c=[[F(rng.randrange(-3,4)) for _ in range(n)] for _ in range(n)]
            a=multiply(transpose(c),c)
            for i in range(n):a[i][i]+=1
            lower,diag=ldlt(a);assert multiply(multiply(lower,diag),transpose(lower))==a
            if n>1:
                for cut in range(1,n):
                    s=schur(a,cut);ls,ds=ldlt(s);assert
multiply(multiply(ls,ds),transpose(ls))==s
            spd_cases+=1
    for m in range(2,8):
```

第28章 矩陣運算：詳解 （續）

```
        for n in range(1,min(4,m+1)):
            for _ in range(12):
                a=[[F(rng.randrange(-3,4)) for _ in range(n)] for _ in range(m)]
                b=[F(rng.randrange(-3,4)) for _ in range(m)]
                try:x,prediction,residual,gram,rhs=least_squares(a,b)
                except ValueError:continue
                assert matvec(transpose(a),residual)==[0]*n
                base=sum(v*v for v in residual)
                for direction in product([-1,0,1],repeat=n):
                    changed=[v+w for v,w in zip(x,direction)]
                    updated=[y-z for y,z in zip(b,matvec(a,changed))]
                    assert sum(v*v for v in updated)==base+sum(v*v for v in
matvec(a,direction))
                least_cases+=1
    a=[[1,0],[1,1],[1,2]];b=[0,2,1]
    assert least_squares(a,b)==([F(1,2),F(1,2)],[F(1,2),1,F(3,2)],
                                [F(-1,2),1,F(-1,2)],[[3,3],[3,5]],[3,4])
    plus=multiply(inverse(multiply(transpose(a),a)),transpose(a))
    assert plus==[[F(5,6),F(1,3),F(-1,6)],[F(-1,2),0,F(1,2)]]
    projection=multiply(a,plus)
    assert multiply(plus,a)==identity(2)
    assert multiply(projection,projection)==projection==transpose(projection)
    assert matvec(projection,b)==[F(1,2),1,F(3,2)]
    chol_a=[[4,2,2],[2,5,1],[2,1,3]]
    assert ldlt(chol_a)==([[1,0,0],[F(1,2),1,0],[F(1,2),0,1]],[[4,0,0],[0,4,0],[0,0,2]])
    assert inverse([[1,1],[2,5]])==[[F(5,3),F(-1,3)],[F(-2,3),F(1,3)]]
    # Without swapping the already-built L prefix, the sample factorization fails.
    wrong_l=[row[:] for row in l];wrong_l[1][0],wrong_l[2][0]=wrong_l[2][0],wrong_l[1][0]
    assert multiply(wrong_l,u)!=[SAMPLE[i] for i in p]
    print(f'PASS exact_small_matrices={count} nonsingular={regular}
sampled_two_sided_inverses={inversions} '
          f'SPD_and_all_leading_Schur_cases={spd_cases}
least_squares_orthogonality_and_Pythagoras={least_cases} '
          'L_prefix_swap_counterexample=True sample_x=1,-1,2 residual_squared=3/2')


if __name__=='__main__':main()
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study